

Operating Systems Lab Assignment: Concurrency with Semaphores, Critical Sections, and Monitor Locks

adam white

October 10, 2026

1 Introduction

This lab had six programs that use threads. Each one uses semaphores, mutexes, or condition variables so the threads don't mess up the data they share.

2 Exercise 1: Producer-Consumer in C

```
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define NUM_ITEMS 10

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t full, empty;
pthread_mutex_t mutex;

void *producer(void *arg) {
    for (int i = 0; i < NUM_ITEMS; i++) {
        int item = rand() % 100;
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = item;
        printf("Produced: %d\n", item);
        in = (in + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        sleep(1);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < NUM_ITEMS; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        sleep(2);
    }
}
```

```

    }
    return NULL;
}

int main() {
    pthread_t prod, cons;
    sem_init(&full, 0, 0);
    sem_init(&empty, 0, BUFFER_SIZE);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&full);
    sem_destroy(&empty);
    pthread_mutex_destroy(&mutex);
    return 0;
}

```

Explanation: The program uses two semaphores and a mutex. `empty` starts at 5 and keeps track of how many open spots are in the buffer, and `full` starts at 0 and keeps track of how many items are in it. Before the producer adds something it waits on `empty`, and when it's done it posts `full`. The consumer does the opposite. The mutex is locked while a thread is changing the buffer so both of them can't be in there at once.

Analysis: Since the producer sleeps for 1 second and the consumer sleeps for 2, the producer gets ahead. By the end the producer was done and the consumer was still taking out the last few items. Everything came out in the same order it went in. If the semaphores weren't there, the producer could write over items that hadn't been read yet and the consumer could read from an empty spot. Taking out the mutex would let both threads change the buffer at the same time.

```

Problems  Output  Debug Console  Terminal  Ports
@whiadam → /workspaces/cs375-labs/cs375-alilab5prod_consumer (main) $ ./producer_consumer
Produced: 83
Consumed: 83
Produced: 86
Consumed: 86
Produced: 77
Produced: 15
Consumed: 77
Produced: 93
Produced: 35
Consumed: 15
Produced: 86
Produced: 92
Consumed: 93
Produced: 49
Produced: 21
Consumed: 35
Consumed: 86
Consumed: 92
Consumed: 49
Consumed: 21
@whiadam → /workspaces/cs375-labs/cs375-alilab5prod_consumer (main) $

```

Figure 1: Compilation and execution of `producer_consumer.c`

3 Exercise 2: Bank Account Simulation in C++

```

#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <vector>
#include <chrono>

class BankAccount {
private:
    int balance;
    std::mutex mtx;
    std::condition_variable cv;
public:
    BankAccount(int initial_balance) : balance(initial_balance) {}

    void deposit(int amount) {
        std::unique_lock<std::mutex> lock(mtx);
        balance += amount;
        std::cout << "Deposited:␣" << amount << " ,␣New␣Balance:␣" << balance <<
            "\n";
        cv.notify_all();
    }

    void withdraw(int amount) {
        std::unique_lock<std::mutex> lock(mtx);
        cv.wait(lock, [&] { return balance >= amount; });
        balance -= amount;
        std::cout << "Withdrew:␣" << amount << " ,␣New␣Balance:␣" << balance <<
            "\n";
    }
};

void user_transactions(BankAccount &account) {
    account.deposit(100);
    std::this_thread::sleep_for(std::chrono::milliseconds(500));
    account.withdraw(50);
}

int main() {
    BankAccount account(100);
    std::vector<std::thread> users;
    for (int i = 0; i < 5; ++i) {
        users.push_back(std::thread(user_transactions, std::ref(account)));
    }
    for (auto &t : users) {
        t.join();
    }
    return 0;
}

```

Explanation: Both deposit and withdraw lock the mutex first, so only one thread can change the balance at a time. withdraw uses cv.wait to wait until there's enough money. When someone deposits, notify_all wakes up anyone who was waiting so they can check the balance again.

Analysis: Five threads each deposit 100 and then withdraw 50. The balance went up to 600 and ended at 350, which is right since $100 + 500 - 250 = 350$. None of the threads had to wait in this run because there was always more than 50 in the account. The wait is there so the balance can't go negative if a withdraw asks for more than what's there. Without the lock, two threads could read the same balance at once and one of the updates would get lost.

4 Exercise 3: Reader-Writer Problem in C



```
Problems Output Debug Console Terminal Ports
@whiadam → /workspaces/cs375-labs/cs375-alilab5prod_consumer (main) $ ./bank_account
Deposited: 100, New Balance: 200
Deposited: 100, New Balance: 300
Deposited: 100, New Balance: 400
Deposited: 100, New Balance: 500
Deposited: 100, New Balance: 600
Withdrew: 50, New Balance: 550
Withdrew: 50, New Balance: 500
Withdrew: 50, New Balance: 450
Withdrew: 50, New Balance: 400
Withdrew: 50, New Balance: 350
@whiadam → /workspaces/cs375-labs/cs375-alilab5prod_consumer (main) $
```

Figure 2: Compilation and execution of bank_account.cpp

```
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <unistd.h>

sem_t rw_mutex, mutex;
int reader_count = 0;
int shared_data = 0;

void *reader(void *arg) {
    sem_wait(&mutex);
    reader_count++;
    if (reader_count == 1) sem_wait(&rw_mutex);
    sem_post(&mutex);

    printf("Reader reads: %d\n", shared_data);
    sleep(1);

    sem_wait(&mutex);
    reader_count--;
    if (reader_count == 0) sem_post(&rw_mutex);
    sem_post(&mutex);
    return NULL;
}

void *writer(void *arg) {
    sem_wait(&rw_mutex);
    shared_data++;
    printf("Writer writes: %d\n", shared_data);
    sleep(1);
    sem_post(&rw_mutex);
    return NULL;
}

int main() {
    pthread_t readers[3], writers[2];
    sem_init(&rw_mutex, 0, 1);
    sem_init(&mutex, 0, 1);

    for (int i = 0; i < 3; i++) pthread_create(&readers[i], NULL, reader, NULL);
    for (int i = 0; i < 2; i++) pthread_create(&writers[i], NULL, writer, NULL);
}
```

```

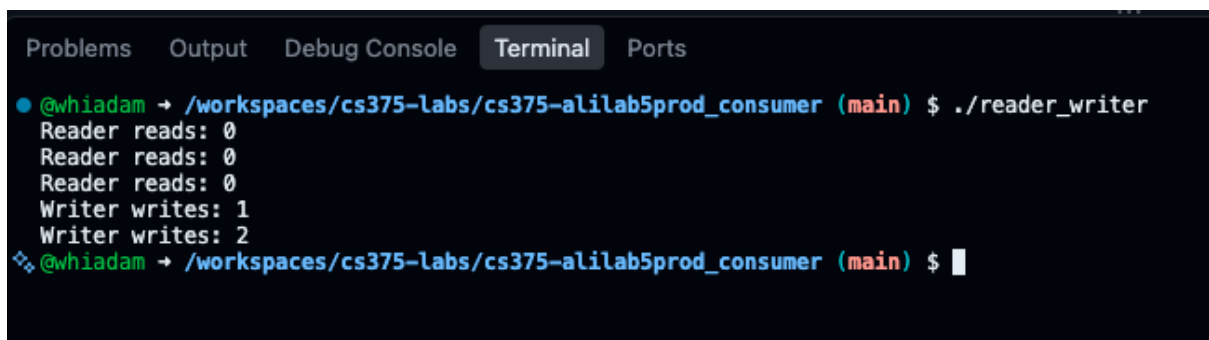
    for (int i = 0; i < 3; i++) pthread_join(readers[i], NULL);
    for (int i = 0; i < 2; i++) pthread_join(writers[i], NULL);

    sem_destroy(&rw_mutex);
    sem_destroy(&mutex);
    return 0;
}

```

Explanation: rw_mutex controls access to the shared data, and mutex just protects reader_count. The first reader to come in waits on rw_mutex, which locks out the writers, and the last reader to leave posts it. A writer has to get rw_mutex by itself, so when a writer is in, nobody else is.

Analysis: This setup gives readers priority. In my output all three readers read 0 at the same time, then the two writers went one after the other and wrote 1 and 2. The downside is that if readers kept coming in nonstop, a writer could end up waiting forever. Without the semaphores a reader could see the data while it's being changed, and two writers doing shared_data++ at once could lose an update.



```

Problems  Output  Debug Console  Terminal  Ports
@whiadam → /workspaces/cs375-labs/cs375-alilab5prod_consumer (main) $ ./reader_writer
Reader reads: 0
Reader reads: 0
Reader reads: 0
Writer writes: 1
Writer writes: 2
@whiadam → /workspaces/cs375-labs/cs375-alilab5prod_consumer (main) $

```

Figure 3: Compilation and execution of reader_writer.c

5 Exercise 4: Dining Philosophers in C++

```

#include <iostream>
#include <thread>
#include <mutex>
#include <vector>
#include <chrono>

#define NUM_PHILOSOPHERS 5

std::mutex forks[NUM_PHILOSOPHERS];

void philosopher(int id) {
    int left = id;
    int right = (id + 1) % NUM_PHILOSOPHERS;
    if (id == NUM_PHILOSOPHERS - 1) { // Avoid deadlock
        std::swap(left, right);
    }

    for (int i = 0; i < 3; ++i) {
        std::this_thread::sleep_for(std::chrono::milliseconds(500));
        forks[left].lock();
        forks[right].lock();
        std::cout << "Philosopher␣" << id << "␣is␣eating.\n";
        std::this_thread::sleep_for(std::chrono::milliseconds(500));
        forks[right].unlock();
        forks[left].unlock();
    }
}

```

```

int main() {
    std::vector<std::thread> philosophers;
    for (int i = 0; i < NUM_PHILOSOPHERS; ++i) {
        philosophers.push_back(std::thread(philosopher, i));
    }
    for (auto &t : philosophers) {
        t.join();
    }
    return 0;
}

```

Explanation: Each fork is a mutex, and a philosopher has to lock two of them to eat. If all five grabbed their left fork at the same time, they would all be stuck waiting for the right one. The code fixes this by having the last philosopher swap the order, so every philosopher picks up the lower numbered fork first. That way the waiting can't go all the way around the table, so there's no deadlock.

Analysis: Each philosopher ate 3 times, so there were 15 lines. In my run the order was 1, 3, 0, 2, 4 every round. Two neighbors never ate at the same time since they share a fork. If the swap was taken out, the program could freeze with everyone holding one fork.

```

Problems  Output  Debug Console  Terminal  Ports
● @whiadam → /workspaces/cs375-labs/cs375-alilab5prod_consumer (main) $ ./dining_philosophers
Philosopher 1 is eating.
Philosopher 3 is eating.
Philosopher 0 is eating.
Philosopher 2 is eating.
Philosopher 4 is eating.
Philosopher 1 is eating.
Philosopher 3 is eating.
Philosopher 0 is eating.
Philosopher 2 is eating.
Philosopher 4 is eating.
Philosopher 1 is eating.
Philosopher 3 is eating.
Philosopher 0 is eating.
Philosopher 2 is eating.
Philosopher 4 is eating.

```

Figure 4: Compilation and execution of dining_philosophers.cpp

6 Exercise 5: Sleeping Barber Problem in C

```

#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <unistd.h>

#define NUM_CHAIRS 3
#define NUM_CUSTOMERS 5

sem_t customers, barber, chairs;
pthread_mutex_t mutex;

void *barber_thread(void *arg) {
    while (1) {
        sem_wait(&customers);
        pthread_mutex_lock(&mutex);
        sem_post(&chairs);
        pthread_mutex_unlock(&mutex);
        printf("Barber is cutting hair.\n");
        sleep(2);
        sem_post(&barber);
    }
}

```

```

    }
    return NULL;
}

void *customer_thread(void *arg) {
    pthread_mutex_lock(&mutex);
    if (sem_trywait(&chairs) == 0) {
        sem_post(&customers);
        pthread_mutex_unlock(&mutex);
        sem_wait(&barber);
        printf("Customer got haircut.\n");
    } else {
        pthread_mutex_unlock(&mutex);
        printf("Customer left (no chairs).\n");
    }
    return NULL;
}

int main() {
    pthread_t barber_t, customers_t[NUM_CUSTOMERS];
    sem_init(&customers, 0, 0);
    sem_init(&barber, 0, 0);
    sem_init(&chairs, 0, NUM_CHAIRS);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&barber_t, NULL, barber_thread, NULL);

    for (int i = 0; i < NUM_CUSTOMERS; i++) {
        pthread_create(&customers_t[i], NULL, customer_thread, NULL);
        sleep(1);
    }

    for (int i = 0; i < NUM_CUSTOMERS; i++) {
        pthread_join(customers_t[i], NULL);
    }

    // Note: Barber thread runs indefinitely; in practice, add termination
    // logic.
    sem_destroy(&customers);
    sem_destroy(&barber);
    sem_destroy(&chairs);
    pthread_mutex_destroy(&mutex);
    return 0;
}

```

Explanation: There are three semaphores. `customers` counts how many people are waiting, and the barber sleeps on it when nobody is there. `chairs` starts at 3 and counts the open chairs. `barber` is how the barber tells a customer the haircut is done. When a customer shows up it tries to take a chair with `sem_trywait`, and if there isn't one it just leaves. If it gets a chair, it posts `customers` to wake up the barber and then waits on `barber`. The mutex keeps two customers from grabbing chairs at the same time.

Analysis: A new customer shows up every second and each haircut takes 2 seconds, but 3 chairs was enough room and all five customers got haircuts. The barber thread is an infinite loop, so it never actually stops. The program still ends because `main` returns after all the customers finish. With no `chairs` semaphore, there would be no limit on how many customers could wait.

7 Exercise 6: Bounded Buffer with Condition Variables in C++

```

#include <iostream>
#include <cstdlib>
#include <thread>

```

```
Problems Output Debug Console Terminal Ports

Barber is cutting hair.
Barber is cutting hair.
Customer got haircut.
Barber is cutting hair.
Customer got haircut.
Barber is cutting hair.
Customer got haircut.
Barber is cutting hair.
Customer got haircut.
Customer got haircut.
Customer got haircut.
@whiadam → /workspaces/cs375-labs/cs375-alilab5prod_consumer (main) $
```

Figure 5: Compilation and execution of sleeping_barber.c

```
#include <mutex>
#include <condition_variable>
#include <queue>
#include <chrono>

#define BUFFER_SIZE 5
#define NUM_ITEMS 10

std::queue<int> buffer;
std::mutex mtx;
std::condition_variable not_full, not_empty;

void producer() {
    for (int i = 0; i < NUM_ITEMS; ++i) {
        std::unique_lock<std::mutex> lock(mtx);
        not_full.wait(lock, [] { return buffer.size() < BUFFER_SIZE; });
        int item = rand() % 100;
        buffer.push(item);
        std::cout << "Produced:␣" << item << "\n";
        not_empty.notify_one();
        lock.unlock();
        std::this_thread::sleep_for(std::chrono::milliseconds(1000));
    }
}

void consumer() {
    for (int i = 0; i < NUM_ITEMS; ++i) {
        std::unique_lock<std::mutex> lock(mtx);
        not_empty.wait(lock, [] { return !buffer.empty(); });
        int item = buffer.front();
        buffer.pop();
        std::cout << "Consumed:␣" << item << "\n";
        not_full.notify_one();
        lock.unlock();
        std::this_thread::sleep_for(std::chrono::milliseconds(2000));
    }
}

int main() {
    std::thread prod(producer);
    std::thread cons(consumer);

    prod.join();
    cons.join();
    return 0;
}
```

Explanation: This is the same producer and consumer problem but with a mutex and two condition variables. The producer waits on `not_full` until the queue has room, adds an item, and calls `notify_one` on `not_empty`. The consumer waits on `not_empty` until there's something in the queue, takes it, and notifies `not_full`. While a thread is waiting the lock gets released, so the other thread can still run.

Analysis: My output was exactly the same as Exercise 1, which makes sense because the sleep times are the same and `rand()` isn't seeded. The difference is how the buffer gets tracked. In Exercise 1 the semaphores hold the counts, and here the condition variables just check the actual size of the queue. I think this version is easier to read since the condition is written right in the code. The one thing that's harder is that you have to hold the lock to wait. For something that's just counting, semaphores are simpler.



```
Problems  Output  Debug Console  Terminal  Ports
@whiadam → /workspaces/cs375-labs/cs375-alilab5prod_consumer (main) $ ./bounded_buffer
Produced: 83
Consumed: 83
Produced: 86
Consumed: 86
Produced: 77
Produced: 15
Consumed: 77
Produced: 93
Produced: 35
Consumed: 15
Produced: 86
Produced: 92
Consumed: 93
Produced: 49
Produced: 21
Consumed: 35
Consumed: 86
Consumed: 92
Consumed: 49
Consumed: 21
@whiadam → /workspaces/cs375-labs/cs375-alilab5prod_consumer (main) $
```

Figure 6: Compilation and execution of `bounded_buffer.cpp`